Informed Consent

XXX UNIVERSITY

Informed Consent for Participants in Research Projects Involving Human Subjects

Title of the Project: Is Reuse All You Need? A Systematic Comparison of Regular Expression Composition Strategies

Investigators:

Author 1 <xxx@xxx.edu>
Author 2 <xxx@xxx.edu>

Research Summary and Request for Consent

Thank you for considering participation in this academic research study. The purpose of this survey is to investigate the preferences and usage patterns of professional software engineers and developers regarding regular expressions (regexes).

- Estimated duration: Approximately 20 minutes
- Eligibility: Participants must have professional software engineering/development experience and prior familiarity with regexes.

Participants will receive compensation in accordance with the payment structure established by the Prolific platform. Please note that the research team reserves the right to withhold compensation if survey responses are identified as automated or insincere.

This study has been reviewed and approved by the Institutional Review Board (IRB) at XXX University.

- All data collected will remain confidential and be stored on a secure server.
- If you have any questions regarding your rights as a participant; you may contact XXX University's IRB at +X XXXXXXXXXX or xxx@xxx.edu.
- For any inquiries related to the study itself, please contact: Author 1 <xxx@xxx.edu>.

By proceeding with the survey, you consent to the use of your anonymized responses for the purposes of scholarly research and publication.

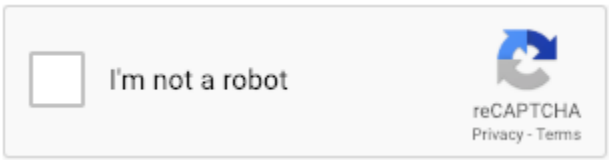
Please continue if you meet **all** of the following conditions:

- You have worked professionally as a software engineer/ developer
- You are familiar with regular expressions (regexes)
- You have used regexes in practice
- You provide informed consent to participate
- **To help us avoid GDPR issues, by proceeding, you confirm that you are not located in the UK or EU.**

To begin the survey, please complete the CAPTCHA below and proceed to the next page.

We sincerely appreciate your time and participation.

CAPTCHA Verification:*



*Even if you do not consent to the survey, you need to complete the CAPTCHA before clicking the "I do not consent" button, due to Qualtrics' limitations.

Detected Prolific Information:

Prolific ID: ${e://Field/PROLIFIC_PID}

Study ID: ${e://Field/STUDY_ID}

Session ID: ${e://Field/SESSION_ID}

Regex Knowledge Check

Screening: Regex Knowledge Check

To proceed with the survey, participants must demonstrate a minimum level of familiarity with regexes by completing the following screening questions.

- The screening consists of **8 multiple-choice questions**.
- You must answer **at least 4 questions correctly** to proceed to the full survey.
- Please spend **no more than 20 seconds per question**

If you do not meet the minimum score, the survey will end at this stage. However, **you will still receive partial compensation for completing the screening section** in accordance with Prolific's policies. Full compensation applies only to participants who qualify and complete the main survey.

Please begin the knowledge check below.

What does the following regular expression match?
`^a+(r|z)\dq*[p-s]$`

◯ A U.S. ZIP code (e.g., 12345 or 12345-6789).

◯ Any string made only of the letter q repeated one or more times (e.g., qqqq).

◯ A string containing only numbers and spaces.

◯ A string that starts with one or more a characters, then either r or z, then a single digit, then zero or more q characters, and ends with one letter from p to s.

What does the following regular expression match?

```
^.+@[^\.].*\.[a-z]{2,}$
```

- A 24-hour time in HH:MM format (e.g., 23:59).
- An IPv4 address in dotted decimal notation (e.g., 192.168.0.1).
- Any URL that starts with http or https.
- Email addresses with any characters before @ and a top-level domain ending in at least two letters.

What does the following regular expression match?

```
^((https?|ftp)\://(([?(\d{1,3}\.){3}\d{1,3}\]?)|(([-a-zA-Z0-9]+\.)+[a-zA-Z]{2,4}))(\:\d+)?(/[-a-zA-Z0-9._?,'+&%$#=~\\]+)*/?)$
```

- A standard IPv6 address in full notation (e.g., 2001:0db8:85a3:0000:0000:8a2e:0370:7334).
- Any string that merely begins with http or https (not necessarily a full URL).
- A Windows file path with drive letter and backslashes (e.g., C:\Users\file.txt).
- A common URL pattern: http/https/ftp followed by ://, then either a hostname (with dots and a 2–4 letter TLD) or an IPv4 address (optionally bracketed), optional :port, and an optional path.

What does the following regular expression match?

```
^(\+|-)?\d+$
```

- Any positive integer only (no sign allowed), like 123.
- A signed integer: an optional + or - followed by one or more digits (e.g., -42, +7, 0).
- A hexadecimal number starting with 0x (e.g., 0x1A3F).
- A single digit between 0 and 9 (e.g., 7).

What does the following regular expression match?

`^[a-zA-Z]$`

- ○ A full sentence ending with a period.
- ○ Any single ASCII letter, uppercase or lowercase (e.g., a or Z).
- ○ Any whole word made of letters of any length (e.g., hello).
- ○ Any single punctuation character (e.g., ! or .).

What does the following regular expression match?

`^((0[1-9])|(1[0-2]))\/(\d{4})$`

- ○ A month/year string in MM/YYYY where the month is 01–12 and the year has exactly 4 digits (e.g., 07/2025).
- ○ A currency amount with dollars and cents (e.g., $123.45).
- ○ A street address like 123 Main St.
- ○ A U.S. phone number in the format (123) 456-7890.

What does the following regular expression match?

`^[2-9]\d{2}-\d{3}-\d{4}$`

- ○ A U.S. phone number in the form NXX-XXX-XXXX where the first digit (N) is 2–9 and the other digits are 0–9, with hyphens (e.g., 212-555-1234).
- ○ A password with uppercase, lowercase, digits, and symbols, like pa$$W0rD!
- ○ A date in MM-DD-YYYY format (e.g., 12-31-2020).
- ○ A floating-point number with optional decimal part (e.g., 3.14).

What does the following regular expression match?
`^#?([a-f]|[A-F]|[0-9]){3}(([a-f]|[A-F]|[0-9]){3})?$`

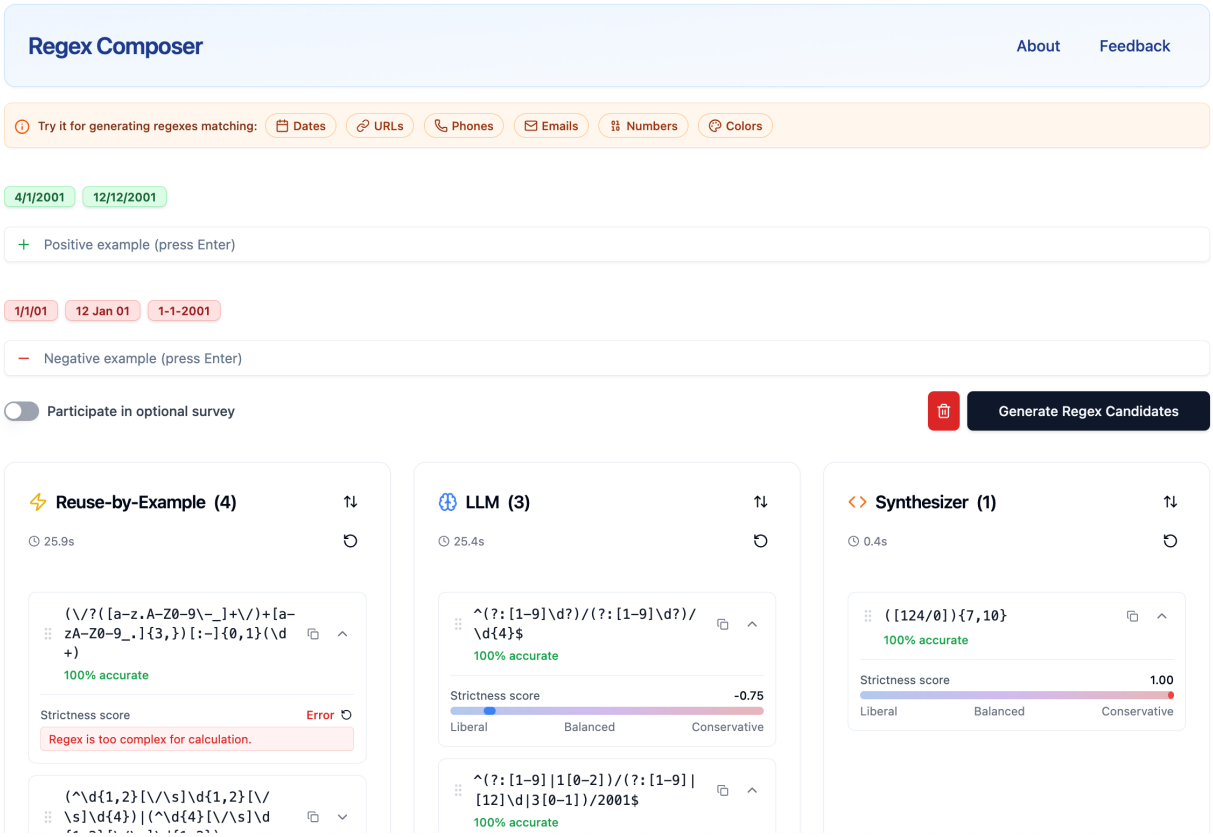
- ○ A CSS hex color optionally beginning with # (e.g., #FF00AA).
- ○ A binary number like 0b101010.
- ○ A C-style hexadecimal literal that starts with 0x followed by one or more hex digits.
- ○ A JavaScript function definition such as "function square(number)".

Tool Information

Welcome to Our Study

What this is:

We are developing a tool that suggests **candidate regex solutions** to developers facing pattern-matching tasks. For each task, the developer provides **positive examples** (strings the regex should match) and **negative examples** (strings it should not). The tool validates candidate regexes against these examples, gathers plausible options from **open-source codebases, internet sources, and auto-composition approaches such as LLMs**, and presents them for review. A screenshot from our tool is given below.



Our tool's interface showing positive/negative examples provided by the developer and suggested candidate regex solutions.

Why are we conducting this survey:

Often, our tool gathers dozens or hundreds of candidate regex solutions, and we are looking for ways to optimize the presentation of solutions to maximize usefulness. One idea we have is that developers may want to see solutions with a lot of variety.

Hence, we want to learn **what amount of variety** in the regex solution candidates you see helps you to understand the pattern-matching task in hand better and decide with more confidence.

What you will do

1. Read the **task description** of an example regex problem with its positive and negative example strings. You will see **four** tasks in total.
2. For each task, review a short list of **individual candidate regex solutions** and **choose one** you would consider to ship.
3. Then you will see **two sets**, each containing **four** valid candidates for the **same** task. The sets differ **only** in **variety (spread of strictness)**. **Choose the set you would use as a reference set.**
4. After answering all four tasks, you will be asked a few quick follow-up questions about the survey and your background.

Definitions

Strictness score: Indicates how **liberal vs. conservative** a regex is relative to the mentioned positive examples. It can take values between -1 and 1. The degree of liberalism increases as the score approaches -1, while the degree of conservatism increases as the score approaches 1.

- **Lower strictness score [-1,0)** → more permissive/matches more beyond the positive examples (i.e., generalizes more).
- **Higher strictness score [0,1]** → more conservative/usually only matches strings that carry similarity to the positive examples (i.e., generalizes less).

Variety (spread): Relates to how **wide or narrow** the strictness scores are **within a presented set of candidate regex solutions.**

- **Narrow spread** → candidate regex solutions in the presented set have similar strictness scores (e.g., most, if not all, of the candidates in the set are either liberal or conservative).

- **Broad spread** → candidate regex solutions in the set presented have varying strictness scores (e.g., the set contains a range of liberal and conservative candidates at the same time).

Reference set: Set of candidate regex solutions you would keep open **side-by-side** before/while finalizing your regex (i.e., the set you would **consult and compare against** to judge whether you are matching **too much** or **too little**).

Before you begin

I understand the concept:

- For each task, I will **first choose one regex that I consider to ship**.
- Then I will **choose a reference set** (not a regex) from **two sets × four candidates**, which differ **only in the spread of strictness** (variety). I will **pick the set** I would keep open to compare candidates and **calibrate whether I am matching too much or too little**.

- ○ No
- ○ Yes

Which statement best describes **variety** in this study?

- ○ The number of candidates in a set
- ○ The spread of strictness (liberal ↔ conservative) within a set
- ○ The visual layout of the tool

Metric Validation 2

Instructions (Task 1/4)

*Instructions are the **same** for Tasks 1–4. If you read the instructions once, you can directly jump to the task description. You can make sense of regexes more easily by clicking on them to view their graphical visualization on Regexper. Additionally, you can experiment with regexes using regex101. Please **do not spend more than three minutes** on each task to answer questions.*

On this page, you will do two things:

1. **Pick one regex** you would ship for a **pattern-matching task** (described below).
2. **Pick a reference set** of candidate regex solutions to consult while deciding (not a regex).

What you will see:

- A small list of **individual candidate regex solutions** for this task. All candidates are **valid**. **You will choose the one** you would consider to ship.
- Then **two sets**, each containing **four** candidate regex solutions for the task. All candidates are **valid**. The sets differ **only** in how much **variety** there is in candidates' **strictness scores**. **You will choose the reference set** you would keep open side-by-side to **compare candidates** and **calibrate whether you are matching too much or too little** before/while picking your single regex.

Definitions:

Strictness score: How **liberal** vs. **conservative** a regex is relative to the mentioned positive examples. Ranging from **-1 to 1**, **lower scores** indicate **greater liberalism**, and **higher scores** indicate **greater conservatism**:

- **Lower strictness score [-1,0)** → more permissive/matches more beyond the positive examples (i.e., generalizes more);
- **Higher strictness score [0,1]** → more conservative/usually only matches strings that carry similarity to the positive examples (i.e., generalizes less).

Variety: The **spread** of strictness scores **within** a set—some sets have a **narrow** spread, others a **wide** spread.
Reference set: A **reference set** is the set of candidates you would keep open **side-by-side** while deciding you are matching **too much** or **too little** (i.e., the set that **our tool would return to assist** you in developing a regex for your task).

Pattern-matching Task Description

Write a regex that matches **decimal IPv4 addresses** in dotted-quad form.

What the regex should achieve

- The string is **exactly four groups** of digits separated by a single dot (**.**).
- Each group is **1–3 digits** and represents a decimal value **from 0 to 255**.
- **Only** dots separate groups—**no** spaces, commas, or other characters.
- **No trailing or leading dot**; **no extra characters** before or after the address.

Positive examples (should match):

- 217.6.9.89
- 0.0.0.0
- 255.255.255.255

Negative examples (should NOT match):

- 256.0.0.0
- 0127.3
- 217.6.9.89.

Questions

Which single regex would you consider to ship for this task?

- ○ Moderately Liberal Strictness Score = -0.10 (25[0-5]|2[0-4]\d|1\d\d|[1-9]?\d)_*(\.(25[0-5]|2[0-4]\d|1\d\d|[1-9]?\d)_*){3}
- ○ Very Conservative Strictness Score = 0.71 ^(?:(?:[1-9]|1\d|2[0-4])?\d|25[0-5])(?:\.(?:(?:[1-9]|1\d|2[0-4])?\d|25[0-5])){3}$
- ○ Moderately Conservative Strictness Score = 0.13 ^((?:(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.){3}(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)(\|?))*$
- ○ Very Liberal Strictness Score = -0.97 ^((25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?).){3}(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)$

How confident are you in your consideration choice?

- ○ Not confident at all
- ○ Slightly confident
- ○ Moderately confident
- ○ Very confident
- ○ Extremely confident

Imagine our tool offers you a set of regexes to use as a reference. Which set do you think would be more helpful to you in composing your regex?

Candidate Regex Solutions – Set A	Candidate Regex Solutions – Set B
Very Liberal — Strictness Score = -0.97 **1.** `^(([1]?[0-9]{1,2}|2([0-4][0-9]|5[0-5])).){3}([1]?[0-9]{1,2}|2([0-4][0-9]|5[0-5]|[a-z]))$`	Very Liberal — Strictness Score = -0.97 **1.** `^((25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?).){3}(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)$`
Very Conservative — Strictness Score = 0.71 **2.** `((\d|[1-9]\d|1\d\d|2[0-4]\d|25[0-5])\.){3}(1\d\d|2[0-4]\d|25[0-5]|[1-9]\d|\d)`	Very Liberal — Strictness Score = -0.52 **2.** `(^localhost$)\|(^((25[0-5]\|(2[0-4]\|1\d\|[1-9]\|)\d)\.?\b){4}$)`
Very Conservative — Strictness Score = 0.71 **3.** `(?:(?:[01]?\d{1,2}|2[0-4]\d|25[0-5])\.){3}(?:[01]?\d{1,2}|2[0-4]\d|25[0-5])`	Moderately Conservative — Strictness Score = 0.13 **3.** `^((?:(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.){3}(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)(|?))*$`
Very Conservative — Strictness Score = 0.71 **4.** `^((\d|[1-9]\d|2[0-4]\d|25[0-5]|1\d\d)(?:\.(\d|[1-9]\d|2[0-4]\d|25[0-5]|1\d\d)){3})$`	Very Conservative — Strictness Score = 0.71 **4.** `^(?:(?:[1-9]|1\d|2[0-4])?\d|25[0-5])(?:\.(?:(?:[1-9]|1\d|2[0-4])?\d|25[0-5])){3}$`

Candidate Regex Solutions - Set A

Candidate Regex Solutions - Set B

In a few sentences, why did you choose this reference set?

Please mention what you compared (e.g., spread of strictness, seeing both liberal and conservative options at the same time, similarities to your initial consideration) and specific aspects you noticed about the sets that influenced your choice.

After reviewing the reference sets, how confident are you in your previous consideration for this task?

- Not confident at all
- Slightly confident
- Moderately confident
- Very confident
- Extremely confident

Would seeing the sets lead you to reconsider your choice?

- No — I would ship my original consideration
- Maybe — I might re-evaluate
- Yes — I would likely change my regex in some way

If switching, which way would you lean?

- ○ To a more liberal regex
- ○ To a more conservative regex
- ○ To a different regex at similar strictness

Metric Validation 3

Instructions (Task 2/4)

*Instructions are the **same** for Tasks 1–4. If you read the instructions once, you can directly jump to the task description. You can make sense of regexes more easily by clicking on them to view their graphical visualization on Regexper. Additionally, you can experiment with regexes using regex101. Please **do not spend more than three minutes** on each task to answer questions.*

On this page, you will do two things:

1. **Pick one regex** you would ship for a **pattern-matching task** (described below).
2. **Pick a reference set** of candidate regex solutions to consult while deciding (not a regex).

What you will see:

- A small list of **individual candidate regex solutions** for this task. All candidates are **valid**. **You will choose the one** you would consider to ship.
- Then **two sets**, each containing **four** candidate regex solutions for the task. All candidates are **valid**. The sets differ **only** in how much

variety there is in candidates' **strictness scores**. **You will choose the reference set** you would keep open side-by-side to **compare candidates** and **calibrate whether you are matching too much or too little** before/while picking your single regex.

Definitions:

Strictness score: How **liberal** vs. **conservative** a regex is relative to the mentioned positive examples. Ranging from **-1 to 1**, **lower scores** indicate **greater liberalism**, and **higher scores** indicate **greater conservatism**:

- **Lower strictness score [-1,0)** → more permissive/matches more beyond the positive examples (i.e., generalizes more);
- **Higher strictness score [0,1]** → more conservative/usually only matches strings that carry similarity to the positive examples (i.e., generalizes less).

Variety: The **spread** of strictness scores **within** a set—some sets have a **narrow** spread, others a **wide** spread.
Reference set: A **reference set** is the set of candidates you would keep open **side-by-side** while deciding you are matching **too much** or **too little** (i.e., the set that **our tool would return to assist** you in developing a regex for your task).

Pattern-matching Task Description

Write a regex that matches decimal numbers, **with or without** a scientific-notation exponent.

What the regex should achieve

- Optional leading **sign**: + or −.
- A **whole number** (e.g., 123) **or** a **decimal with one dot** and digits on **both** sides (e.g., 123.35).
- Optional **exponent** part: [eE] followed by an optional sign and **at least one digit** (e.g., e−2).
- **No letters** other than e/E in the exponent marker.
- **No multiple dots**, **no missing exponent digits**, **no extra characters**.

Positive examples (should match):

- 123
- -123.35
- -123.35e-2

Negative examples (should NOT match):

- abc
- 123.32e
- 123.32.3

Questions

Which single regex would you consider to ship for this task?

- ○ Moderately Conservative Strictness Score = 0.22 `^(-?\d+)(?:\.(\d*))?(?:e(-?\d+))?$`
- ○ Moderately Conservative Strictness Score = 0.05 `^-?\d+(?:\.\d+)?(?:e[+-]?\d+)?`
- ○ Very Liberal Strictness Score = -0.54 `(")|((-?)(\b0[xX][a-fA-F0-9]+|(\b\d+(\.\d*)?|\.\d+)([eE][-+]?\d+)?))|({}|(\[]|(\/\/)|(\/\*)|(,|\])`
- ○ Moderately Liberal Strictness Score = -0.31 `^([\-\+])?(0*)(\.[0-9]+([eE]\-?[0-9]+)?|[0-9]+(\.[0-9]+([eE]\-?[0-9]+)?)?)$`

How confident are you in your consideration choice?

- ○ Not confident at all
- ○ Slightly confident
- ○ Moderately confident
- ○ Very confident
- ○ Extremely confident

Imagine the tool offers you a set of regexes to use as a reference. Which set do you think would be more helpful to you in composing your regex?

Candidate Regex Solutions – Set A	Candidate Regex Solutions – Set B
Moderately Liberal — Strictness Score = -0.04 **1.** `([+-]?[0-9]+(?:\.[0-9]+)?)(?:([eEsSfFdDlL])([+-]?[0-9]+))?`	Very Liberal — Strictness Score = -0.54 **1.** `(")|((-?)(\b0[xX][a-fA-F0-9]+|(\b\d+(\.\d*)?|\.\d+)([eE][-+]?\d+)?))|({)|(\[)|(\/\/)|(\/\*)|(,|\])`
Moderately Conservative — Strictness Score = 0.05 **2.** `^-?[0-9]+(?:\.[0-9]+)?(?:[eE](?:-|\+)?[0-9]+)?$`	Moderately Liberal — Strictness Score = -0.31 **2.** `^([\-\+])?(0*)(\.[0-9]+([eE]\-?[0-9]+)?|[0-9]+(\.[0-9]+([eE]\-?[0-9]+)?)?)$`
Moderately Conservative — Strictness Score = 0.05 **3.** `-?([0-9]*\.)?[0-9]+([eE]-?[0-9]+)?`	Moderately Conservative — Strictness Score = 0.00 **3.** `^-?[1-9]\d*\.?(\d+([eE][-+]\d+)?)?$`
Moderately Conservative — Strictness Score = 0.05 **4.** `-?\b\d+(?:\.\d+)?(?:[eE][-+]?\d+)?\b`	Moderately Conservative — Strictness Score = 0.05 **4.** `^-?\d+(?:\.\d+)?(?:e[+-]?\d+)?`

Candidate Regex Solutions - Set A ○ Candidate Regex Solutions - Set B ○

In a few sentences, why did you choose this reference set?

Please mention what you compared (e.g., spread of strictness, seeing both

liberal and conservative options at the same time, similarities to your initial consideration) and specific aspects you noticed about the sets that influenced your choice.

After reviewing the reference sets, how confident are you in your previous consideration for this task?

- ○ Not confident at all
- ○ Slightly confident
- ○ Moderately confident
- ○ Very confident
- ○ Extremely confident

Would seeing the sets lead you to reconsider your choice?

- ○ No — I would ship my original consideration
- ○ Maybe — I might re-evaluate
- ○ Yes — I would likely change my regex in some way

If switching, which way would you lean?

- ○ To a more liberal regex
- ○ To a more conservative regex
- ○ To a different regex at similar strictness

Metric Validation 4

Instructions (Task 3/4)

*Instructions are the **same** for Tasks 1–4. If you read the instructions once, you can directly jump to the task description. You can make sense of regexes more easily by clicking on them to view their graphical visualization on Regexper. Additionally, you can experiment with regexes using regex101. Please **do not spend more than three minutes** on each task to answer questions.*

On this page, you will do two things:

1. **Pick one regex** you would ship for a **pattern-matching task** (described below).
2. **Pick a reference set** of candidate regex solutions to consult while deciding (not a regex).

What you will see:

- A small list of **individual candidate regex solutions** for this task. All candidates are **valid**. **You will choose the one** you would consider to ship.
- Then **two sets**, each containing **four** candidate regex solutions for the task. All candidates are **valid**. The sets differ **only** in how much **variety** there is in candidates' **strictness scores**. **You will choose the reference set** you would keep open side-by-side to **compare candidates** and **calibrate whether you are matching too much or too little** before/while picking your single regex.

Definitions:

Strictness score: How **liberal** vs. **conservative** a regex is relative to the mentioned positive examples. Ranging from **-1 to 1**, **lower scores** indicate **greater liberalism**, and **higher scores** indicate **greater conservatism**:

- **Lower strictness score [-1,0)** → more permissive/matches more beyond the positive examples (i.e., generalizes more);
- **Higher strictness score [0,1]** → more conservative/usually only matches strings that carry similarity to the positive examples (i.e., generalizes less).

Variety: The **spread** of strictness scores **within** a set—some sets have a **narrow** spread, others a **wide** spread.
Reference set: A **reference set** is the set of candidates you would keep open **side-by-side** while deciding you are matching **too much** or **too little** (i.e., the set that **our tool would return to assist** you in developing a regex for your task).

Pattern-matching Task Description

Write a regex that validates **UK National Insurance Numbers (NINo)** in compact form.

What the regex should achieve

- Format: **two letters**, **six digits**, optional **final letter**.
- First **two letters** must be from **A–Z excluding** `D, F, I, Q, U, V` (case-sensitive).
- The **six digits** are `0`–9 (no separators).
- If a **final letter** is present, it must be **A, B, C, D, F, or M**.

- **No spaces or punctuation** anywhere.

Positive examples (should match):

- JG103759A
- AP019283D
- ZX047829C

Negative examples (should NOT match):

- DC135798A
- FQ987654C
- KL192845T

Questions

1) Which single regex would you consider to ship for this task?

- ◯ Very Liberal Strictness Score = -1.00 ([^0*#,\.]*)([0#,\.]+)([^0*#,\.]*)$
- ◯ Very Liberal Strictness Score = -0.60 \w+0\w*
- ◯ Very Conservative Strictness Score = 0.80 ^[A-CEGHJ-PR-TW-Z]{1}[A-CEGHJ-NPR-TW-Z]{1}[0-9]{6}[A-DFM]{0,1}$
- ◯ Moderately Conservative Strictness Score = 0.06 ^[A-CEGHJ-PR-TW-Z][A-CEGHJ-NPR-TW-Z] ?[0-9]{2} ?[0-9]{2} ?[0-9]{2} ?[ABCD]?

How confident are you in your consideration choice?

- ○ Not confident at all
- ○ Slightly confident
- ○ Moderately confident
- ○ Very confident
- ○ Extremely confident

2) Imagine the tool offers you a set of regexes to use as a reference. Which set do you think would be more helpful to you in composing your regex?

Candidate Regex Solutions – Set A	Candidate Regex Solutions – Set B
Very Liberal — Strictness Score = −1.00 **1.** `([^0]*)0+(.*)`	Very Liberal — Strictness Score = −1.00 **1.** `([^0*#,\.]*)([0#,\.]+)([^0*#,\.]*)$`
Very Liberal — Strictness Score = −0.99 **2.** `([^\+\-]*(?:0|0\*|\*0){1,}[^\+\-]*)|(?:1\*|\*1)`	Very Liberal — Strictness Score = −1.00 **2.** `^([1-9][0-9]+\.[0-9]{2}$)\|^([1-9]+\.[0-9]{2}$)|(^[^0]*0[^0]*)$`
Very Liberal — Strictness Score = −0.99 **3.** `.*0(\d).*`	Moderately Conservative — Strictness Score = 0.06 **3.** `^[A-CEGHJ-PR-TW-Z][A-CEGHJ-NPR-TW-Z] ?[0-9]{2} ?[0-9]{2} ?[0-9]{2} ?[ABCD]?`
Very Liberal — Strictness Score = −0.71 **4.** `\w+0\w+`	Very Conservative — Strictness Score = 0.80 **4.** `^[A-CEGHJ-PR-TW-Z]{1}[A-CEGHJ-NPR-TW-Z]{1}[0-9]{6}[A-DFM]{0,1}$`

Candidate Regex Solutions - Set A ○ Candidate Regex Solutions - Set B ○

In a few sentences, why did you choose this reference set?

Please mention what you compared (e.g., spread of strictness, seeing both liberal and conservative options at the same time, similarities to your initial

consideration) and specific aspects you noticed about the sets that influenced your choice.

After reviewing the reference sets, how confident are you in your previous consideration for this task?

- ○ Not confident at all
- ○ Slightly confident
- ○ Moderately confident
- ○ Very confident
- ○ Extremely confident

Would seeing the sets lead you to reconsider your choice?

- ○ No — I would ship my original consideration
- ○ Maybe — I might re-evaluate
- ○ Yes — I would likely change my regex in some way

If switching, which way would you lean?

- ○ To a more liberal regex
- ○ To a more conservative regex
- ○ To a different regex at similar strictness

Metric Validation 5

Instructions (Task 4/4)

*Instructions are the **same** for Tasks 1–4. If you read the instructions once, you can directly jump to the task description. You can make sense of regexes more easily by clicking on them to view their graphical visualization on Regexper. Additionally, you can experiment with regexes using regex101. Please **do not spend more than three minutes** on each task to answer questions.*

On this page, you will do two things:

1. **Pick one regex** you would ship for a **pattern-matching task** (described below).
2. **Pick a reference set** of candidate regex solutions to consult while deciding (not a regex).

What you will see:

- A small list of **individual candidate regex solutions** for this task. All candidates are **valid**. **You will choose the one** you would consider to ship.
- Then **two sets**, each containing **four** candidate regex solutions for the task. All candidates are **valid**. The sets differ **only** in how much **variety** there is in candidates' **strictness scores**. **You will choose the reference set** you would keep open side-by-side to **compare candidates** and **calibrate whether you are matching too much or too little** before/while picking your single regex.

Definitions:

Strictness score: How **liberal** vs. **conservative** a regex is relative to the mentioned positive examples. Ranging from **-1 to 1**, **lower scores** indicate

greater liberalism, and **higher scores** indicate **greater conservatism**:

- **Lower strictness score [-1,0)** → more permissive/matches more beyond the positive examples (i.e., generalizes more);
- **Higher strictness score [0,1]** → more conservative/usually only matches strings that carry similarity to the positive examples (i.e., generalizes less).

Variety: The **spread** of strictness scores **within** a set—some sets have a **narrow** spread, others a **wide** spread.
Reference set: A **reference set** is the set of candidates you would keep open **side-by-side** while deciding you are matching **too much** or **too little** (i.e., the set that **our tool would return to assist** you in developing a regex for your task).

Pattern-matching Task Description

Write a regex that matches dates of the form **M(M)/D(D)/YYYY** using slashes.

What the regex should achieve

- **Month** and **day** are **1 or 2 digits** each.
- **Year** is **exactly 4 digits**.
- The separator is **"/"** only.
- **Do not** enforce calendar validity (e.g., month 55 is still "format-valid" for this task).
- **Do not** allow other formats (hyphens, text months, two-digit years, etc.).

Positive examples (should match):

- 4/1/2001
- 12/12/2001
- 55/5/3434

Negative examples (should NOT match):

- 1/1/01
- 12 Jan 01
- 1-1-2001

Questions

Which single regex would you consider to ship for this task?

- Moderately Liberal Strictness Score = -0.17 `^\d{1,2}\/\d{1,2}\/\d{4}(, \d{1,2}:\d{2}:\d{1,2} [A|P]M)?$`
- Moderately Conservative Strictness Score = 0.09 `([0-9]{1,2}\/){2}[0-9]{4}( [0-9]{1,2}(:[0-9]{2}){2})?`
- Very Liberal Strictness Score = -0.97 `^[a-zA-Z0-9\\s\\t_#&\\-\\/\\.,\\(\\):\\{\\}_\\\\!;]{8,}$`
- Very Conservative Strictness Score = 1.00 `^\d{1,2}/\d{1,2}/\d{4}$`

How confident are you in your consideration choice?

- ○ Not confident at all
- ○ Slightly confident
- ○ Moderately confident
- ○ Very confident
- ○ Extremely confident

Imagine the tool offers you a set of regexes to use as a reference. Which set do you think would be more helpful to you in composing your regex?

Candidate Regex Solutions – Set A	Candidate Regex Solutions – Set B
Very Conservative — Strictness Score = 0.50 **1.** `0?(\d{1,2})\/0?(\d{1,2})\/(\d{4})`	Very Liberal — Strictness Score = -0.99 **1.** `([^-]+[0-9]{4})`
Very Conservative — Strictness Score = 1.00 **2.** `\d{1,2}/\d{1,2}/\d{4}`	Moderately Liberal — Strictness Score = -0.17 **2.** `^\d{1,2}\/\d{1,2}\/\d{4}(, \d{1,2}:\d{2}:\d{1,2} [A|P]M)?$`
Very Conservative — Strictness Score = 1.00 **3.** `(?:0?(\d)|(\d{2}))\/(?:0?(\d)|(\d{2}))\/(\d{4})`	Very Conservative — Strictness Score = 0.85 **3.** `^(\d{1,2})(?:\/(\d{1,2}))?\/(\d{4})$`
Very Conservative — Strictness Score = 1.00 **4.** `\b\d{1,2}\/\d{1,2}\/\d{4}\b`	Very Conservative — Strictness Score = 1.00 **4.** `^[0-9]{1,2}/[0-9]{1,2}/[0-9]{4}`

Candidate Regex Solutions - Set A ○ Candidate Regex Solutions - Set B ○

In a few sentences, why did you choose this reference set?

Please mention what you compared (e.g., spread of strictness, seeing both liberal and conservative options at the same time, similarities to your initial consideration) and specific aspects you noticed about the sets that influenced your choice.

After reviewing the reference sets, how confident are you in your previous consideration for this task?

- ○ Not confident at all
- ○ Slightly confident
- ○ Moderately confident
- ○ Very confident
- ○ Extremely confident

Would seeing the sets lead you to reconsider your choice?

- ○ No — I would ship my original consideration
- ○ Maybe — I might re-evaluate
- ○ Yes — I would likely change my regex in some way

If switching, which way would you lean?

- ○ To a more liberal regex
- ○ To a more conservative regex
- ○ To a different regex at similar strictness

Metric Validation 6

Instructions (Task 5/4)

*Instructions are the **same** for Tasks 1–4. If you read the instructions once, you can directly jump to the task description. You can make sense of regexes more easily by clicking on them to view their graphical visualization on Regexper. Additionally, you can experiment with regexes using regex101. Please **do not spend more than three minutes** on each task to answer questions.*

On this page, you will do two things:

1. **Pick one regex** you would ship for a **pattern-matching task** (described below).
2. **Pick a reference set** of candidate regex solutions to consult while deciding (not a regex).

What you will see:

- A small list of **individual candidate regex solutions** for this task. All candidates are **valid**. **You will choose the one** you would consider to ship.
- Then **two sets**, each containing **four** candidate regex solutions for the task. All candidates are **valid**. The sets differ **only** in how much **variety** there is in candidates' **strictness scores**. **You will choose the reference set** you would keep open side-by-side to **compare candidates** and **calibrate whether you are matching too much or too little** before/while picking your single regex.

Definitions:

Strictness score: How **liberal** vs. **conservative** a regex is relative to the mentioned positive examples. Ranging from **-1 to 1**, **lower scores** indicate **greater liberalism**, and **higher scores** indicate **greater conservatism**:

- **Lower strictness score [-1,0)** → more permissive/matches more beyond the positive examples (i.e., generalizes more);
- **Higher strictness score [0,1]** → more conservative/usually only matches strings that carry similarity to the positive examples (i.e., generalizes less).

Variety: The **spread** of strictness scores **within** a set—some sets have a **narrow** spread, others a **wide** spread.
Reference set: A **reference set** is the set of candidates you would keep open **side-by-side** while deciding you are matching **too much** or **too little** (i.e., the set that **our tool would return to assist** you in developing a regex for your task).

Pattern-matching Task Description

Write a regex that matches common **U.S. 10-digit phone numbers** with flexible formatting.

What the regex should achieve

- Accept **exactly 10 digits** (no country code, no extensions).
- Allow these formats:
  - **No delimiters:** `2225551212`
  - **Optional parentheses** around area code: `(222) 555 1212`, `(222) 555–1212`
  - **Optional separators** between groups: single **space** or **dash** (e.g., `222 555 1212`, `222–555–1212`)
- Enforce simple North American Numbering Plan (NANP) digit rules:
  - **Area code** and **central office code** cannot be in the range **001–199** (i.e., their **first digit is 2–9**).

Positive examples (should match):

- 5305551212
- (530) 555-1212
- 530-555-1212

Negative examples (should NOT match):

- 0010011212
- 1991991212
- 123) not-good

Questions

Which single regex would you consider to ship for this task?

- Very Liberal Strictness Score = -0.99 ^([1-9][0-9]+\.[0-9]{2}$)|^([1-9]+\.[0-9]{2}$)|(^[^0]*0[^0]*)$
- Very Conservative Strictness Score = 0.58 ^[(]?[2-9][0-9]{2}[)]?[-]?[0-9]{3}[-]?[0-9]{4}$
- Moderately Liberal Strictness Score = -0.12 ^((\+1|1)?(|-)?)?(\([2-9][0-9]{2}\)|[2-9][0-9]{2})(|-)?([2-9][0-9]{2}(|-)?[0-9]{4})$
- Moderately Conservative Strictness Score = 0.31 ^\(?([2-9][0-8][0-9])\)?[-.]?([2-9][0-9]{2})[-.]?([0-9]{4})$

How confident are you in your consideration choice?

- ○ Not confident at all
- ○ Slightly confident
- ○ Moderately confident
- ○ Very confident
- ○ Extremely confident

Imagine the tool offers you a set of regexes to use as a reference. Which set do you think would be more helpful to you in composing your regex?

Candidate Regex Solutions – Set A	Candidate Regex Solutions – Set B
Moderately Liberal — Strictness Score = -0.38 **1.** `\(?(?<areacode>[1]?[2-9]\d{2})\)?[\s-]?(?<prefix>[2-9]\d{2})[\s-]?(?<linenumber>[\d]{4})`	Very Liberal — Strictness Score = -0.99 **1.** `.*([A-Za-z0-9])\1\1.*`
Moderately Liberal — Strictness Score = -0.21 **2.** `^[01]?[- .]?\(?[2-9]\d{2}\)?[- .]?\d{3}[- .]?\d{4}$`	Very Liberal — Strictness Score = -0.99 **2.** `^([1-9][0-9]+\.[0-9]{2}$)\|^([1-9]+\.[0-9]{2}$)|(^[^0]*0[^0]*)$`
Moderately Liberal — Strictness Score = -0.19 **3.** `(\([2-9]\d\d\)|[2-9]\d\d) ?[-.,]? ?[2-9]\d\d ?[-.,]? ?\d{4}`	Moderately Conservative — Strictness Score = 0.31 **3.** `^\(?([2-9][0-8][0-9])\)?[-. ]?([2-9][0-9]{2})[-. ]?([0-9]{4})$`
Moderately Liberal — Strictness Score = -0.12 **4.** `^((\+1|1)?( |-)?)?(\([2-9][0-9]{2}\)|[2-9][0-9]{2})( |-)?([2-9][0-9]{2}( |-)?[0-9]{4})$`	Very Conservative — Strictness Score = 0.58 **4.** `^[(]?[2-9][0-9]{2}[)]?[ -]?[0-9]{3}[ -]?[0-9]{4}$`

Candidate Regex Solutions - Set A ○ Candidate Regex Solutions - Set B ○

In a few sentences, why did you choose this reference set?

Please mention what you compared (e.g., spread of strictness, seeing both liberal and conservative options at the same time, similarities to your initial consideration) and specific aspects you noticed about the sets that influenced your choice.

After reviewing the reference sets, how confident are you in your previous consideration for this task?

- ○ Not confident at all
- ○ Slightly confident
- ○ Moderately confident
- ○ Very confident
- ○ Extremely confident

Would seeing the sets lead you to reconsider your choice?

- ○ No — I would ship my original consideration
- ○ Maybe — I might re-evaluate
- ○ Yes — I would likely change my regex in some way

If switching, which way would you lean?

- ○ To a more liberal regex
- ○ To a more conservative regex
- ○ To a different regex at similar strictness

Metric Validation 7

Instructions (Task 6/4)

*Instructions are the **same** for Tasks 1–4. If you read the instructions once, you can directly jump to the task description. You can make sense of regexes more easily by clicking on them to view their graphical visualization on Regexper. Additionally, you can experiment with regexes using regex101. Please **do not spend more than three minutes** on each task to answer questions.*

On this page, you will do two things:

1. **Pick one regex** you would ship for a **pattern-matching task** (described below).
2. **Pick a reference set** of candidate regex solutions to consult while deciding (not a regex).

What you will see:

- A small list of **individual candidate regex solutions** for this task. All candidates are **valid**. **You will choose the one** you would consider to ship.
- Then **two sets**, each containing **four** candidate regex solutions for the task. All candidates are **valid**. The sets differ **only** in how much **variety** there is in candidates' **strictness scores**. **You will choose the reference set** you would keep open side-by-side to **compare candidates** and **calibrate whether you are matching too much or too little** before/while picking your single regex.

Definitions:

Strictness score: How **liberal** vs. **conservative** a regex is relative to the mentioned positive examples. Ranging from **-1 to 1**, **lower scores** indicate **greater liberalism**, and **higher scores** indicate **greater conservatism**:

- **Lower strictness score [-1,0)** → more permissive/matches more beyond the positive examples (i.e., generalizes more);
- **Higher strictness score [0,1]** → more conservative/usually only matches strings that carry similarity to the positive examples (i.e., generalizes less).

Variety: The **spread** of strictness scores **within** a set—some sets have a **narrow** spread, others a **wide** spread.
Reference set: A **reference set** is the set of candidates you would keep open **side-by-side** while deciding you are matching **too much** or **too little** (i.e., the set that **our tool would return to assist** you in developing a regex for your task).

Pattern-matching Task Description

Write a regex that matches **hexadecimal integer literals** of the form used in C-style code: `0x` or `0X` followed by one or more hex digits.

What the regex should achieve

- Required prefix: `0x` **or** `0X`.
- Then **≥1** hexadecimal digit: `0`–`9`, `a`–`f`, or `A`–`F`.
- **No sign**, **no separators/underscores**, **no decimal point**, **no type suffixes** (e.g., u, L, UL), and **no spaces**.

Positive examples (should match):

- 0x0ffe
- 0XBEAF
- 0x1e

Negative examples (should NOT match):

- x0ffe
- 0xG1
- 0xFFu

Questions

1) Which single regex would you consider to ship for this task?

- Moderately Conservative Strictness Score = 0.17 ^(?:(0[Xx][\dA-Fa-f]+))
- Moderately Liberal Strictness Score = -0.12 ^-?(0([Xx][0-9A-Fa-f]+|[0-7]*)|[1-9][0-9]*)
- Very Liberal Strictness Score = -0.73 (?:((?:(?:(?:[1-9]\d*)|(?:0))|(?:0[oO]?[0-7]+)|(?:0[xX][\dA-Fa-f]+)|(?:0[bB][01]+))\b)|(.))
- Very Conservative Strictness Score = 0.50 0[xX][0-9a-fA-F]*

How confident are you in your consideration choice?

- ○ Not confident at all
- ○ Slightly confident
- ○ Moderately confident
- ○ Very confident
- ○ Extremely confident

2) Imagine the tool offers you a set of regexes to use as a reference. Which set do you think would be more helpful to you in composing your regex?

Candidate Regex Solutions – Set A	Candidate Regex Solutions – Set B
Very Liberal — Strictness Score = −0.82 **1.** `("")|((-?)(\b0[xX][a-fA-F0-9]+|(\b\d+(\.\d*)?|\.\d+)([eE][-+]?\d+)?))|({)|(\[)|(\/\/)|(\/\*)|(\S)`	Moderately Conservative — Strictness Score = 0.17 **1.** `\b0[xX][0-9a-fA-F]+\b`
Very Liberal — Strictness Score = −0.73 **2.** `(?:((?:(?:(?:[1-9]\d*)|(?:0))|(?:0[oO]?[0-7]+)|(?:0[xX][\dA-Fa-f]+)|(?:0[bB][01]+))\b)|(.))`	Moderately Conservative — Strictness Score = 0.17 **2.** `^(?:([0][xX]([0-9a-fA-F])+))`
Moderately Conservative — Strictness Score = 0.17 **3.** `^(?:(0[Xx][\dA-Fa-f]+))`	Moderately Conservative — Strictness Score = 0.20 **3.** `(0[xX])?[0-9a-fA-F]*`
Very Conservative — Strictness Score = 0.50 **4.** `0[xX][0-9a-fA-F]*`	Moderately Conservative — Strictness Score = 0.33 **4.** `\b(?:0[bBxX]?|[1-9])[0-9a-fA-F]*$`

Candidate Regex Solutions - Set A ○ Candidate Regex Solutions - Set B ○

In a few sentences, why did you choose this reference set?

Please mention what you compared (e.g., spread of strictness, seeing both liberal and conservative options at the same time, similarities to your initial consideration) and specific aspects you noticed about the sets that influenced your choice.

After reviewing the reference sets, how confident are you in your previous consideration for this task?

- ○ Not confident at all
- ○ Slightly confident
- ○ Moderately confident
- ○ Very confident
- ○ Extremely confident

Would seeing the sets lead you to reconsider your choice?

- ○ No — I would ship my original consideration
- ○ Maybe — I might re-evaluate
- ○ Yes — I would likely change my regex in some way

If switching, which way would you lean?

- ○ To a more liberal regex
- ○ To a more conservative regex
- ○ To a different regex at similar strictness

General Questions

In each task, you first picked the regex you would like to ship. Which aspects did you consider when you made that initial choice? Select all that apply.

- [ ] Strictness near what I believed was right (not too permissive, not too conservative)
- [ ] Likely to generalize to unseen cases
- [ ] Simplicity/short length
- [ ] Readability/maintainability (clear structure, easy to modify)
- [ ] Familiar pattern idioms I trust
- [ ] Use of anchors/boundaries to control scope (^ , $, \b, etc.)
- [ ] Explicit handling of separators/whitespace/punctuation
- [ ] Performance safety (low backtracking risk)
- [ ] Engine portability (works in my target runtime(s))
- [ ] Reusability/adaptability (easy to tune later)
- [ ] Useful captures (named/non-capturing groups set up for extraction)
- [ ] Avoids obscure/fragile constructs (e.g., lookbehind, backreferences)
- [ ] Aligns with team/style guidelines
- [ ] Other (please specify):

Did seeing multiple candidate regex solutions for the same task make you feel more confident in understanding and selecting a regex for the problem?

- ○ No
- ○ Yes

Why didn't viewing multiple candidates increase your confidence?

- [ ] I was already confident in my initial pick
- [ ] The candidates felt too similar (not enough variety)
- [ ] The candidates felt too spread out (too much variety to compare)
- [ ] wanted both extremes represented (very permissive & very conservative)
- [ ] wanted more options near the middle (finer steps around my target)
- [ ] "Strictness"/"variety" wasn't clear enough to me
- [ ] Hard to judge generalization beyond the given examples
- [ ] needed more evidence/metrics (e.g., precision/recall on hidden tests)
- [ ] needed explanations or diffs showing how candidates differ
- [ ] Comparing multiple candidates slowed me down
- [ ] Other (please specify):

After reviewing multiple candidate regex solutions in different reference sets, which of the following changes or additions would have increased your confidence in choosing a regex to ship for this task?

- [ ] More variety (wider spread of strictness)
- [ ] Less variety (narrower spread)
- [ ] More example strings / edge cases
- [ ] Accuracy metrics beyond the shown examples
- [ ] Indicators of likely over-match / under-match tendencies
- [ ] Visual diffs/explanations of each regex
- [ ] Performance/backtracking risk indicator
- [ ] Engine compatibility checks (JS/PCRE/etc.)
- [ ] Ability to test custom inputs in place
- [ ] Other (please specify):

Did seeing multiple candidate regexes with varying strictness levels help you decide which regex to choose for the given regex composition tasks with confidence?

- ○ No
- ○ Yes

Which spread pattern did you look for when choosing the sets?

- ○ More variety (wider spread of strictness)
- ○ Less variety (narrower spread of strictness)
- ○ Not sure

Which forms of “variety” matter most to you when choosing a regex among alternatives? Select all that apply.

- [ ] Strictness spread (permissive ↔ conservative)
- [ ] Extreme anchors (including both very permissive and very conservative options)
- [ ] Local granularity (several nearby variants around a promising strictness level)
- [ ] Structural approaches (different regex strategies—anchors, character classes, alternation, lookarounds, named groups)
- [ ] Readability/maintainability differences (e.g., length, grouping style, clarity)
- [ ] Complexity/performance differences (length, backtracking risk)
- [ ] Edge-case policy differences (how optional separators/whitespace are handled when allowed)
- [ ] Other (please specify):

Free-form

From your perspective as a software engineer/developer, what features would make such a regex composition tool useful in your workflow? Answer in a few sentences.

Do you have any other comments or suggestions for such a regex composition tool?

Demographics

Participant Background

To support our analysis of regex preferences among software engineers/ developers, we ask that you provide some brief information about your professional and educational background. Your responses will help us better understand the context in which different developers use regexes and will be analyzed in aggregate only. All information you provide will remain confidential.

How long have you worked as a professional software developer?

- ○ Less than one year
- ○ 1-2 years
- ○ 3-5 years
- ○ 6-10 years
- ○ More than 10 years

What is your current role?

Please choose an option from the list below that best describes your role.

- ○ Frontend developer
- ○ Backend developer
- ○ Full-stack developer
- ○ Systems administrator
- ○ Mobile developer
- ○ Network engineer
- ○ DevOps engineer
- ○ Team lead
- ○ Tester
- ○ Engineering manager
- ○ Prefer not to say
- ○ Other (please specify): _____

Do you have an academic degree in a computing-related subject?

- ○ No
- ○ Yes
- ○ Prefer not to say

What is the size of the company you most recently worked at as a professional developer?

- ○ Small (100 employees or less)
- ○ Medium (101 - 999 employees)
- ○ Large (1000 employees or more)

Which programming language(s) that support regexes do you have the most experience with?

Please list up to three languages separated by commas.

Estimate your regex expertise level:

- ○ Novice. For example, you know what repetition operators like * and + d
- ○ Intermediate. For example, you have used more sophisticated features like non-greedy quantifiers /a+?/ and character classes / \d | \w | [abc] | [^\d]/.
- ○ Expert. For example, you have used features like backreferences /(a+) \1/ and look-ahead/behind assertions /(?<=abc)def/
- ○ Master. You have written a regular expression engine

Estimate the last time you used regexes in a professional context:

- ○ In the last week
- ○ In the last month
- ○ In the last year
- ○ More than a year ago

End of Survey Block

Thank you for taking part in this study. Please click the next button below to be redirected back to Prolitic and register your submission.